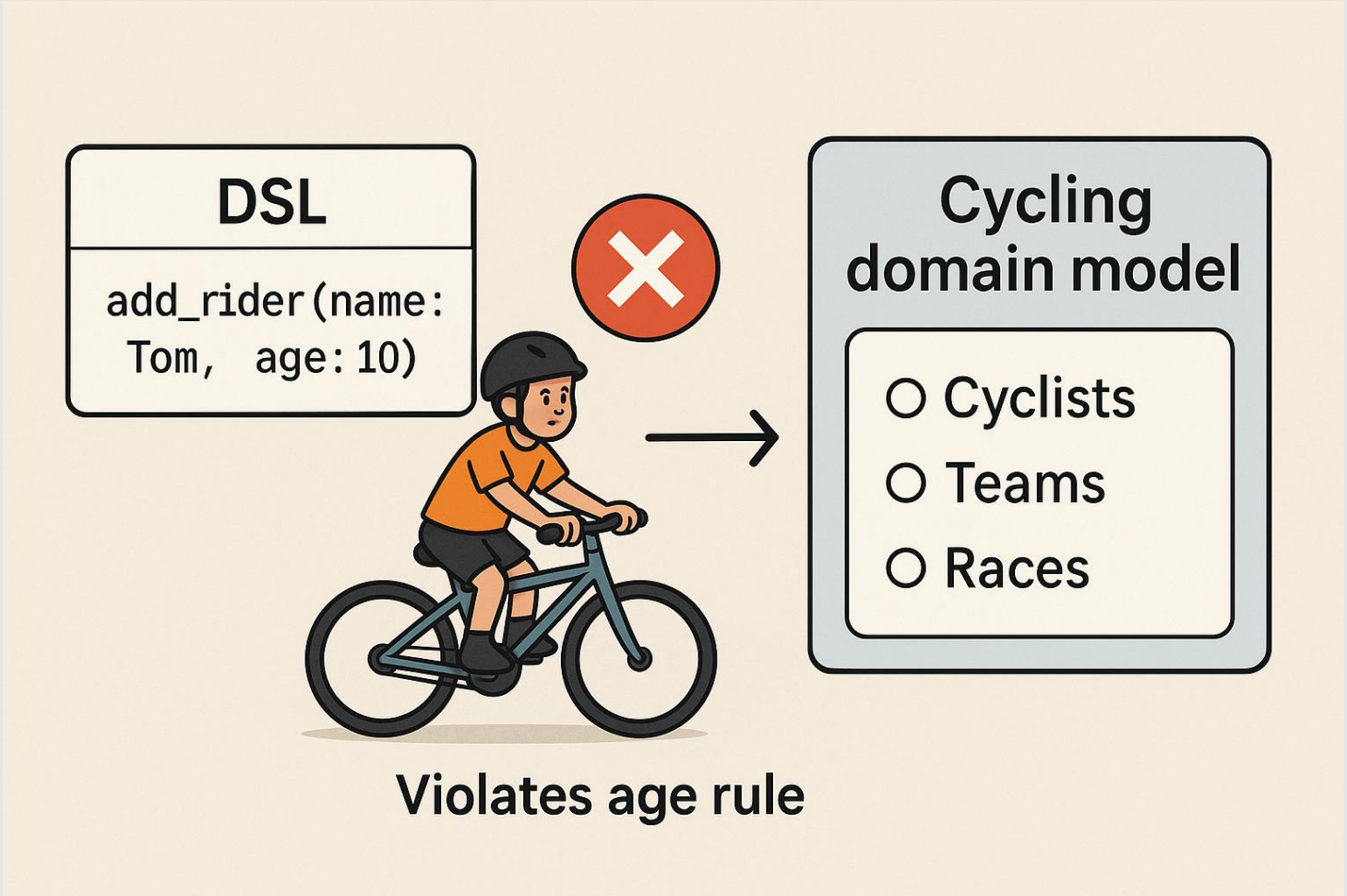
MAKING DSLS CONVERSABLE

Krishna Narasimhan, F1RE ignite change

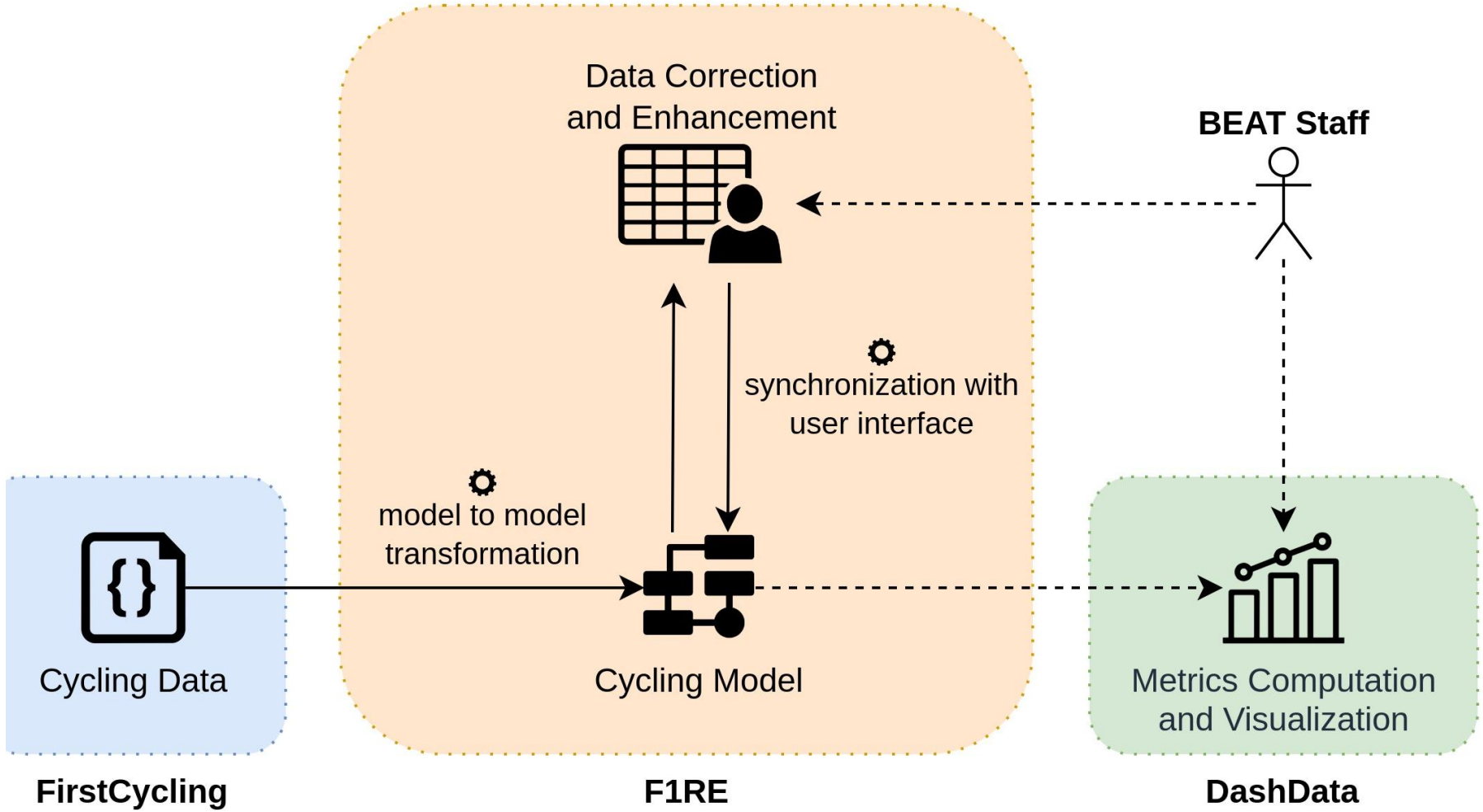
DSLs are great

- DSLs model complex domains with precision.
- They ensure valid, rule-consistent operations.
- Example: cycling domain model enforcing age limits or team structure.



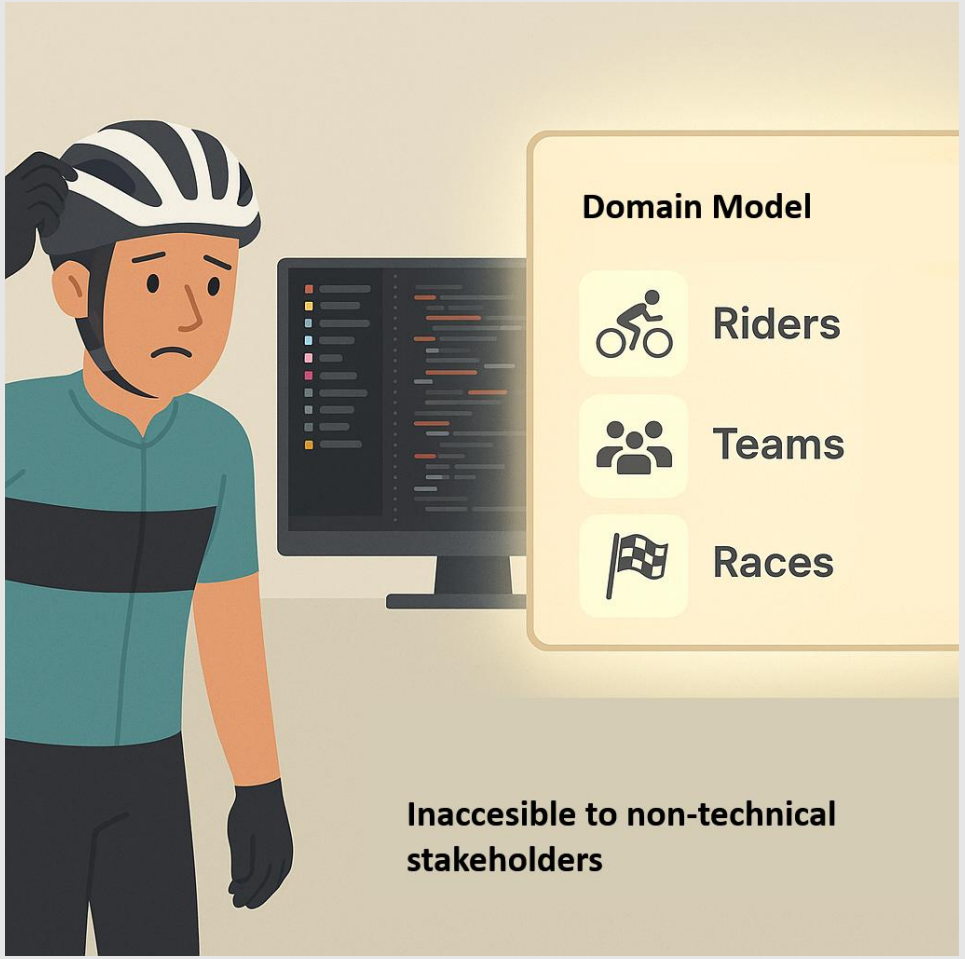
Use case:
BEAT Cyling Club

- Talent prediction for cycling pro’s
- Race results combined with secondary data (weather, location, training data, equipment)
- Problem:
- Users without ANY tech skills
- Users have very limited time
- Fuzzy input


Data Correction
and Enhancement
BEAT Staff
synchronization with
user interface
model to model
transformation
Cycling Data
Cycling Model
Metrics Computation
and Visualization
FirstCycling
F1RE
DashData

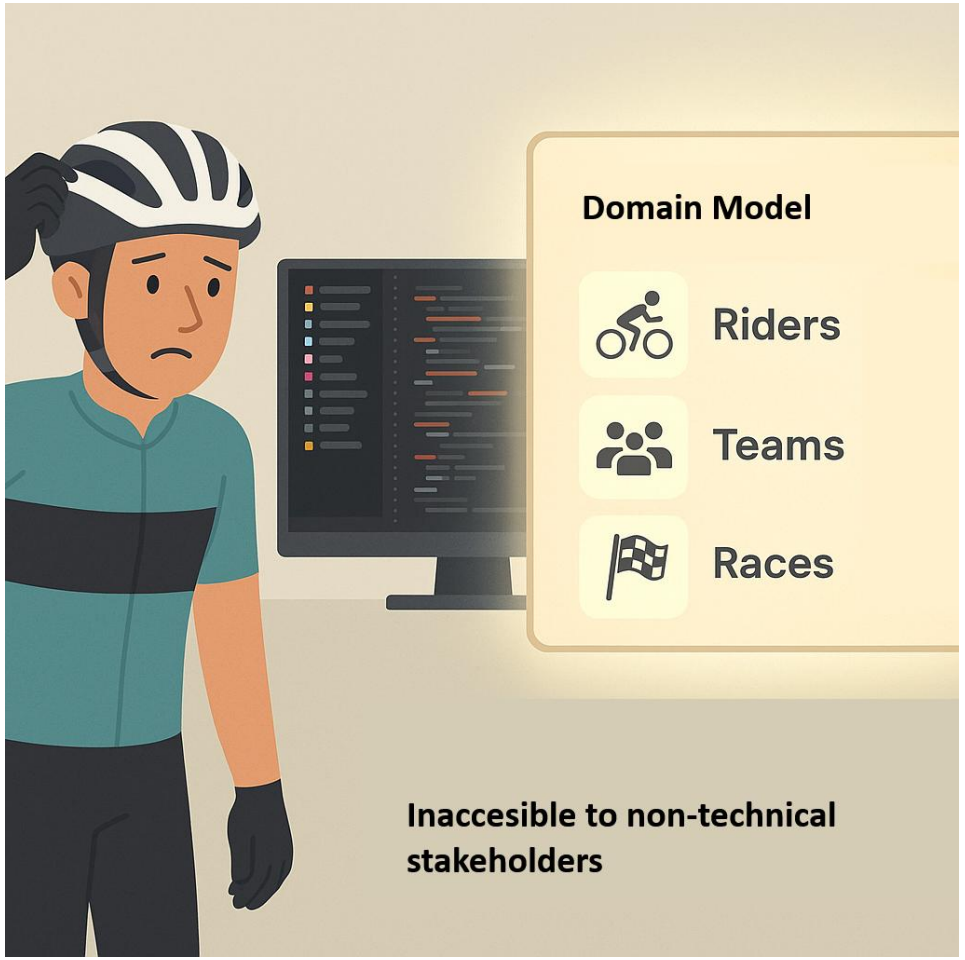
Powerful model, hard to use

- We built a DSL-based model using race and rider data from FirstCycling.com
- The model lets them simulate team composition and season strategies
- But the BEAT staff aren’t techies - the DSL and its tools were too complex to use directly
- They needed a conversational interface: a way to “talk” to the model instead of coding



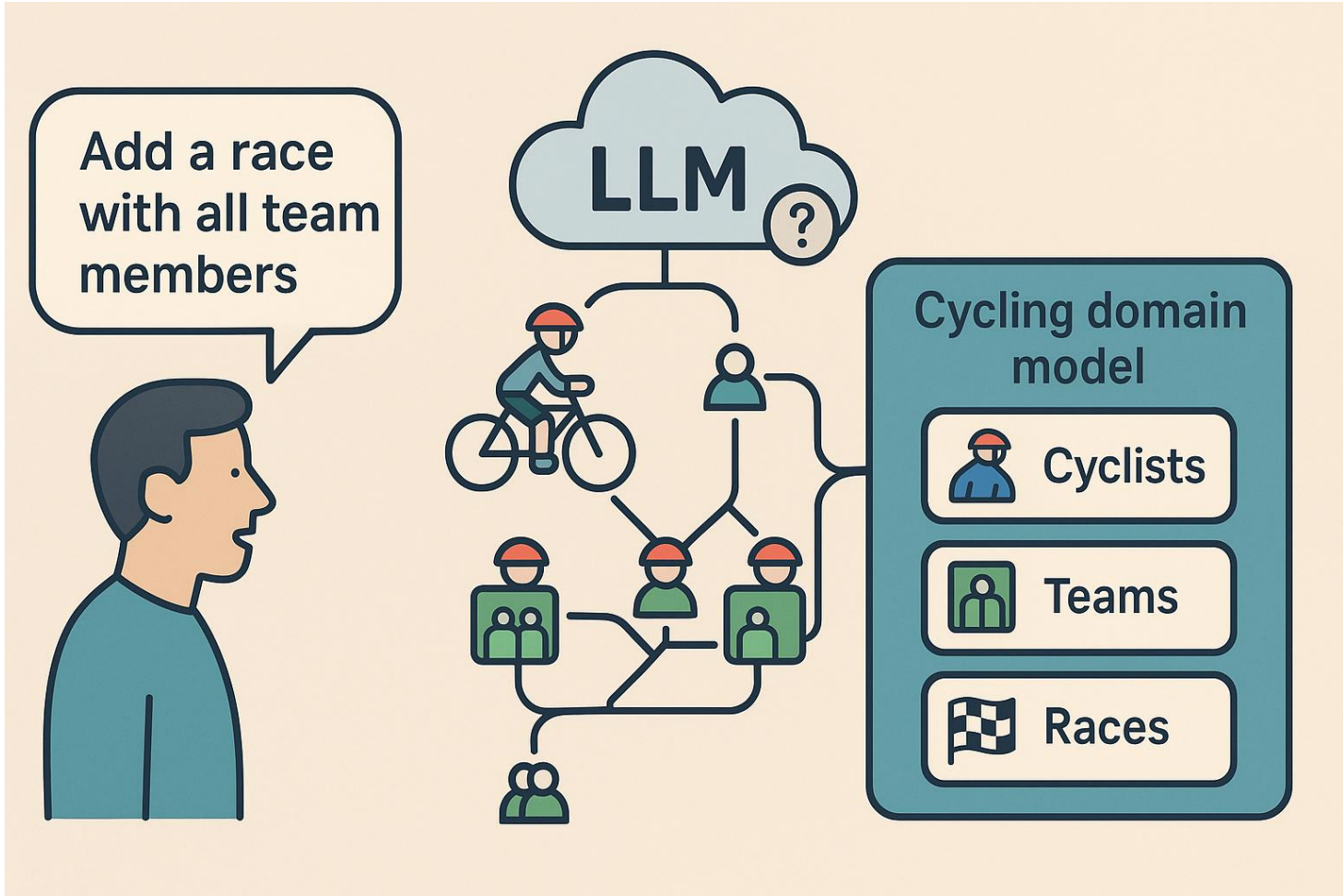
Accessibility issue

- Complex syntax
- Special tooling required
- Limited documentation
- Steep learning curve



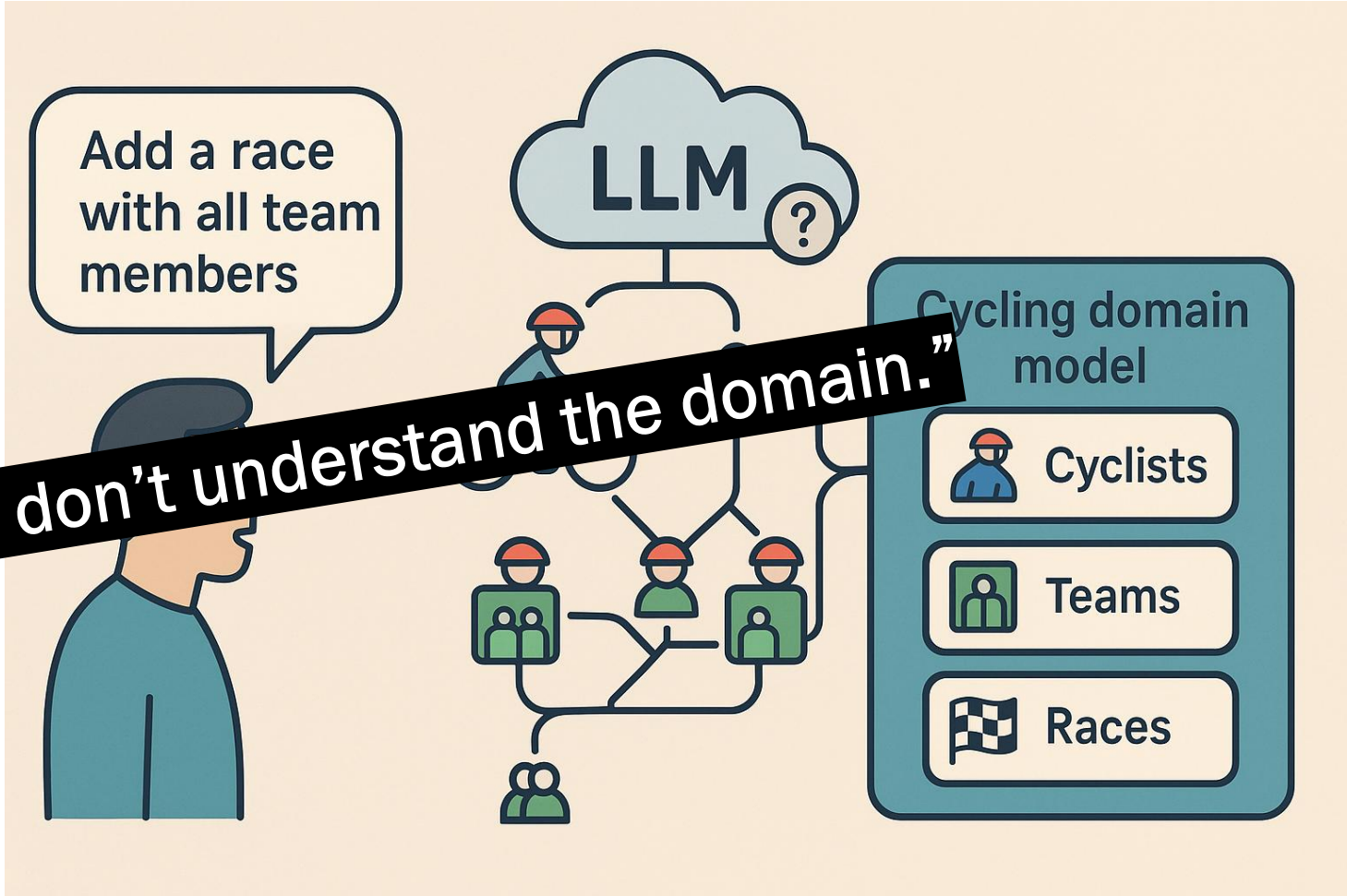
LLMs ≠ Domain Understanding

- Words, not structure
- Confuses relationships
- No rule awareness
- Sounds right, acts wrong



LLMs ≠ Domain Understanding

- Understands words, not domain structure
- Can confuse entities and relationships
- No schema or rule
- S wrong
- Example: “Add a race with all team members” → mismatched or duplicate entries

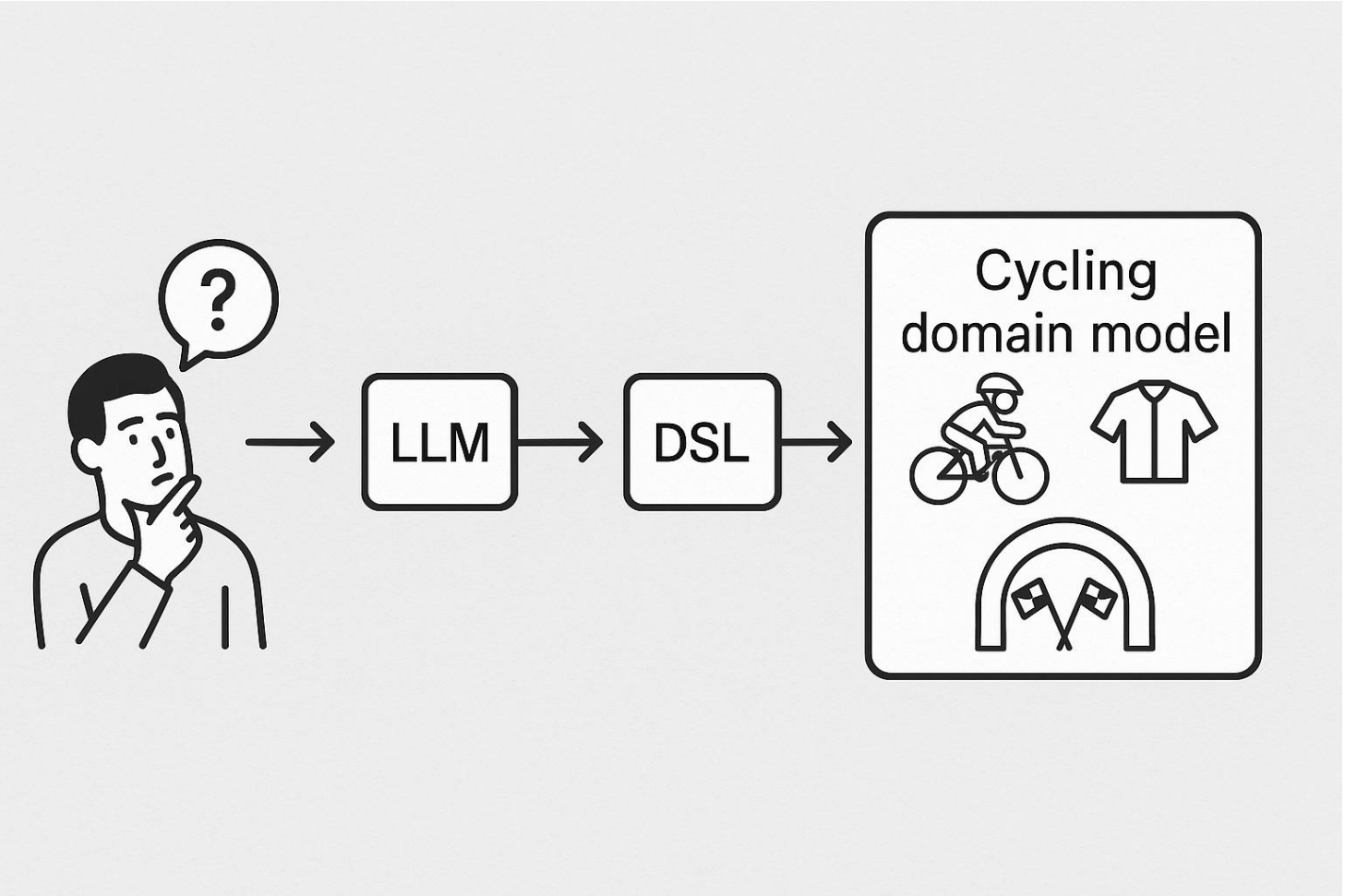


Can we combine them?

- LLMs ⟶ accessible, natural, but unreliable
- DSLs ⟶ reliable, structured, but hard to access
- Can we get both — natural input and trusted execution?
- What would that system look like?

Best of both worlds – Separation of concerns

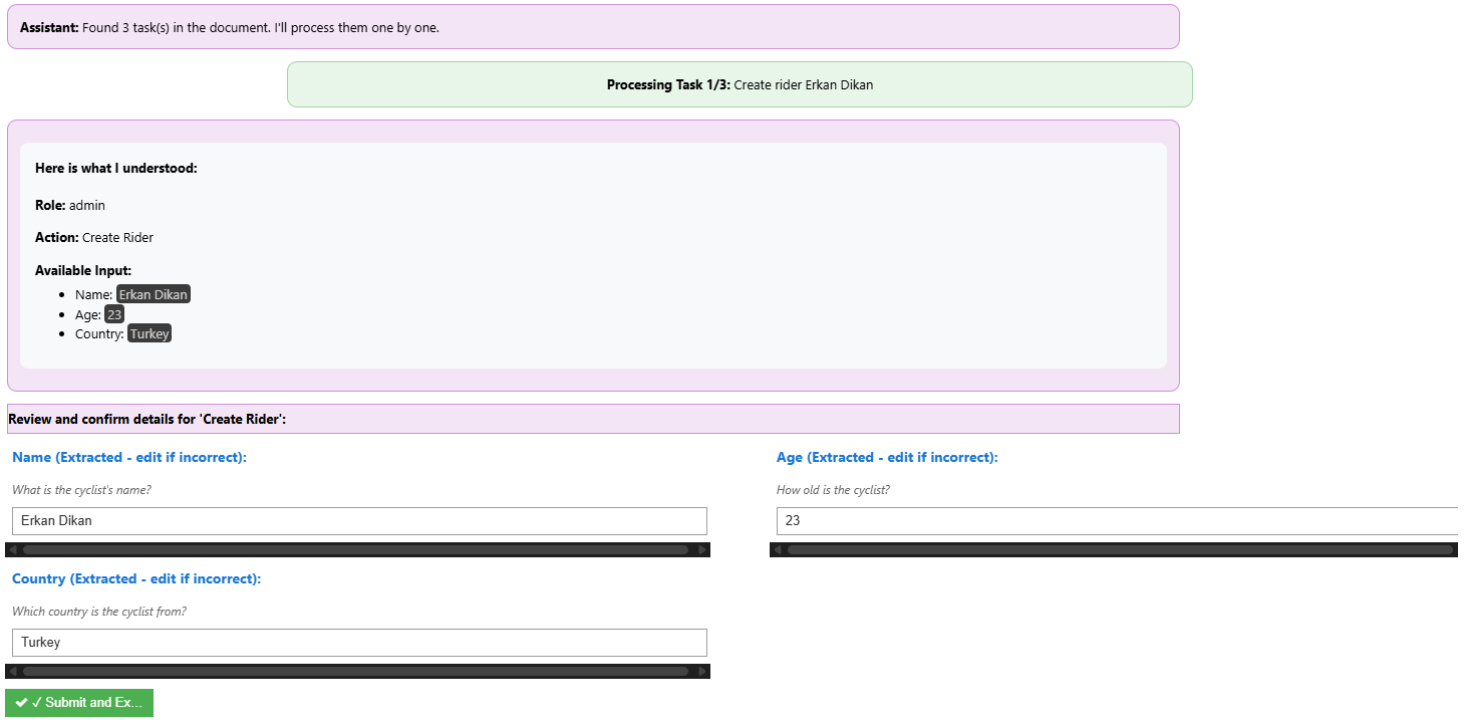
- LLMs offer accessibility — natural, flexible, easy for anyone to use
- DSLs offer reliability — structured, validated, rule-bound
- What if each did what it's best at?
- LLMs understand intent ⟶ DSLs ensure correct execution
- Together: Conversational DSLs — natural input with guaranteed validity



Demo (first part - to show how the envisioned system could look like)

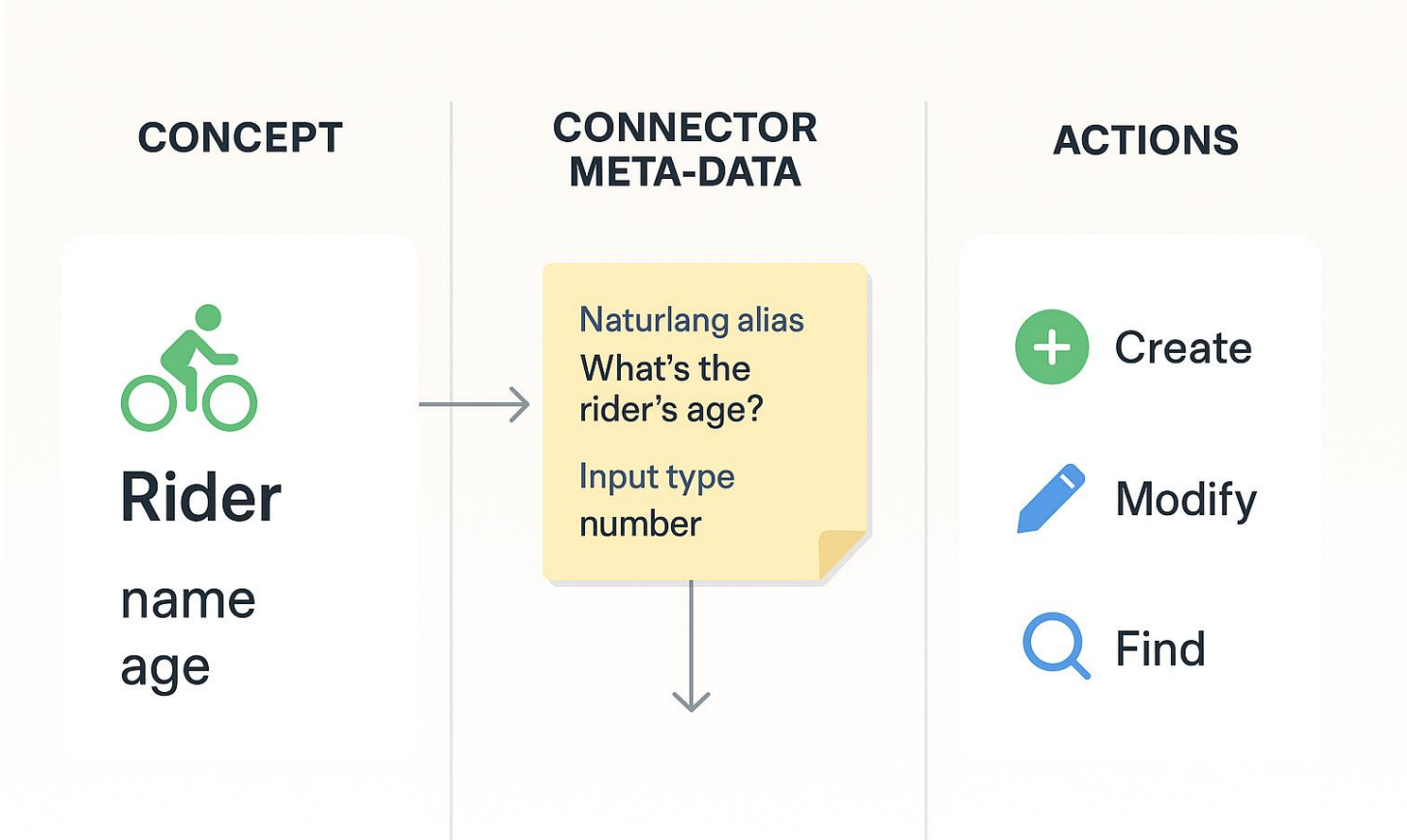
- Making the point “If we could use natural language to interact with the underlying model, this is what it would look like”.
- Show how an input like this results in model change

““Let's set up the cycling team for next season. Create a rider named Erkan Dikan from Turkey aged 23. Create another rider Priya from Germany. Also create Team SkyIN from India with a budget of 20 million founded in 2010.”



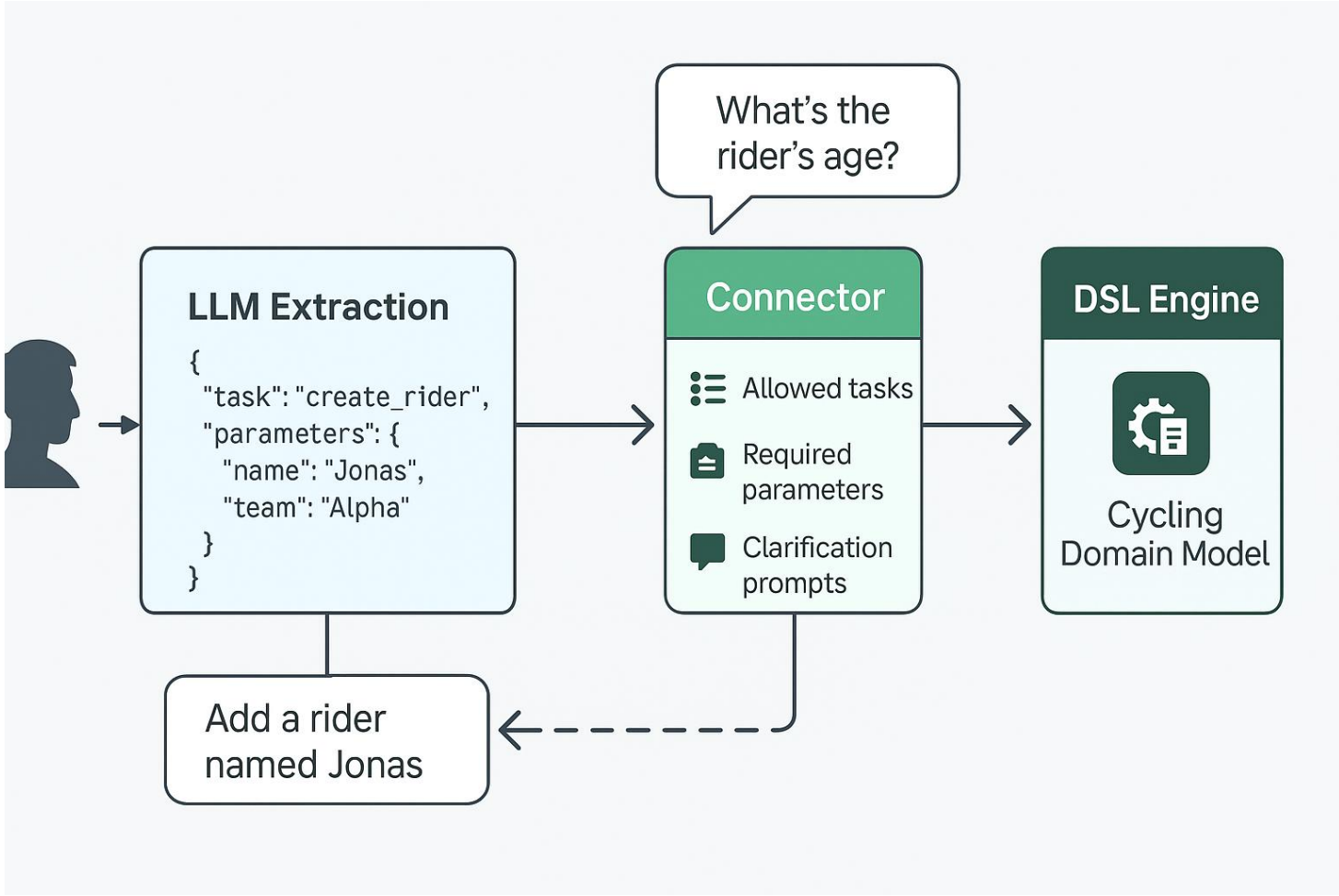
Keeping roles separate with Connector

- LLM never touches domain
- DSL defines all rules
- Connector bridges the gap.



High level system design

- LLM extracts intent
- Connector validates
- Missing info ⟶ clarification
- DSL executes valid actions
- Domain model = source of truth



Implementation

- Two reference implementations
- LionWeb (https://lionweb.io/)
- Lark (https://lionweb.io/)

- Supported model hubs:
- Hugging Face
- Ollama Cloud
- Ollama locally hosted

Demo part 2

- Show multiple queries (create, modify, find) in both cycling (lionweb) and event management (lark)